JIG

Inventory Management System

Technical Documentation

By:

Elijah L. Boon

Chaz B. Honrada

April 2026

Project Overview

Overview

Current Jig Department operations rely on manual tracking of specialized tools and materials, leading to persistent stock inaccuracies and restricted data accessibility for staff. JIG IMS is a web-based Inventory Management System (IMS) designed to centralize inventory data and support dynamic database schema updates. By providing a user-friendly interface, this system will digitize department workflows and ensure real-time oversight of all specialized assets.

The Solution

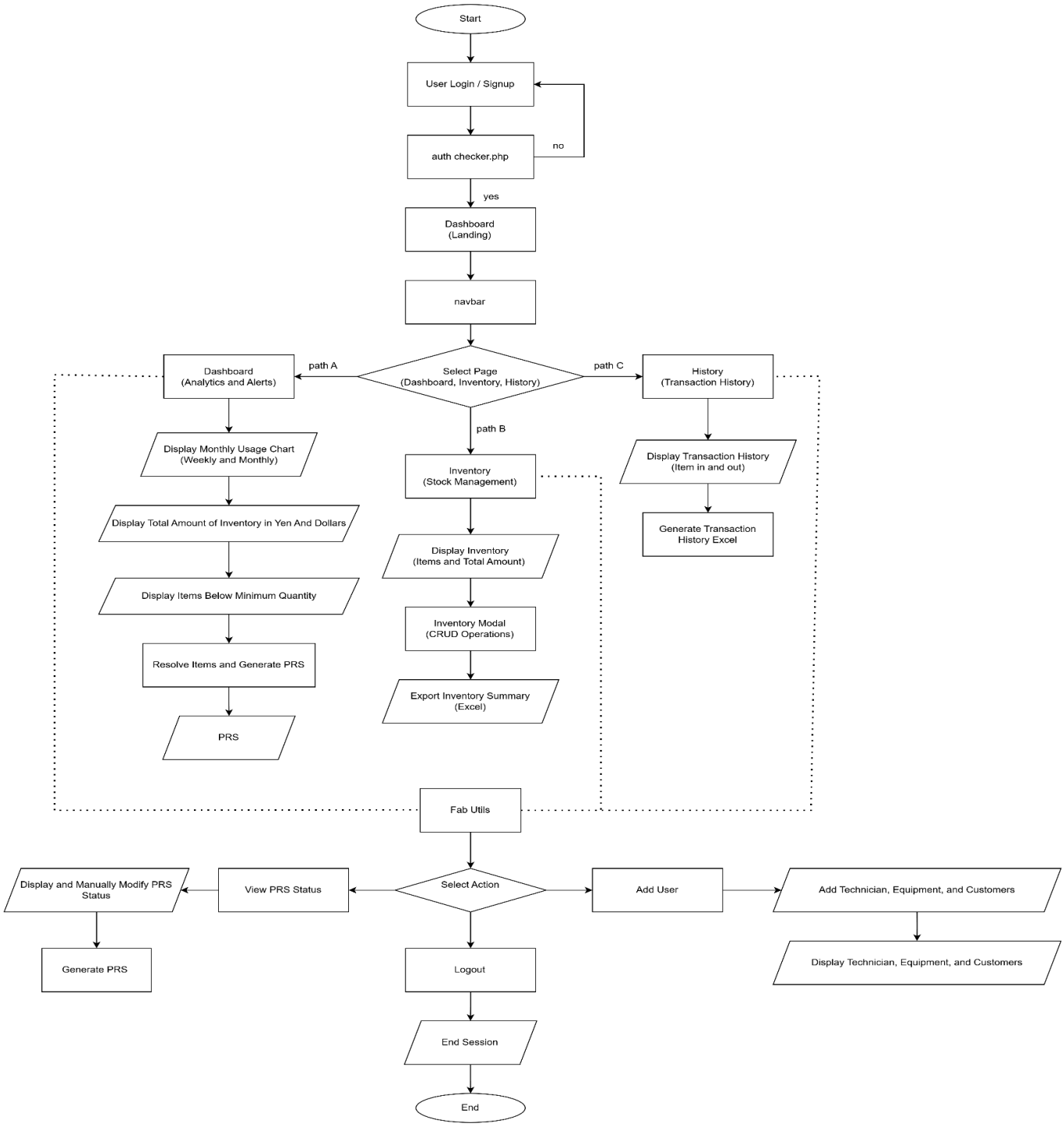
SwiftStock leverages IoT barcode integration and an automated reorder engine to eliminate manual entry errors. By transitioning to a centralized database, the system provides a "single source of truth" accessible across the warehouse, sales, and procurement departments.

Key Capabilities **(to improve)**

- **Real-Time Tracking:** Instant updates on stock levels via mobile QR Code scanning and updating in inventory.
- **Automated Procurement:** Predictive logic that generates purchase request forms when an item hits minimum quantity.
- **Role-Based Security:** Tiered access to ensure data integrity and financial privacy.

Technical Documentation

Flow Chart



Tech Stack

This is a practical, high-performance choice for internal department tools where rapid deployment and dynamic data handling are the top priorities.

- **Frontend:** HTML5, CSS3, and JavaScript (Standard web technologies used to build a responsive, user-friendly interface).
- **Backend:** PHP (Server-side logic featuring a dynamic database handler to adapt to schema changes automatically).
- **Database:** MySQL via XAMPP (A reliable relational database used for centralized storage of specialized tool and material records).
- **Synchronization:** AJAX (Enables 'Sync Changes' functionality for seamless bulk updates without requiring a full page reload).
- **PhpSpreadsheet (^2.1):** The core engine used to resolve inventory data and generate professional **Purchase Request (PR)** reports in `.xlsx` format.
- **Composer:** Utilized for secure dependency management and PSR-4 compliant autoloading.
- **Material Symbols:** Integrated for a modern, intuitive iconography system across the dashboard.

Best for: Small-to-medium departments, rapid internal tool development, and environments requiring flexible, evolving database structures.

System Architecture

The system follows a **WAMP** architecture (Windows, Apache, MySQL, PHP) implemented through the **XAMPP** stack.

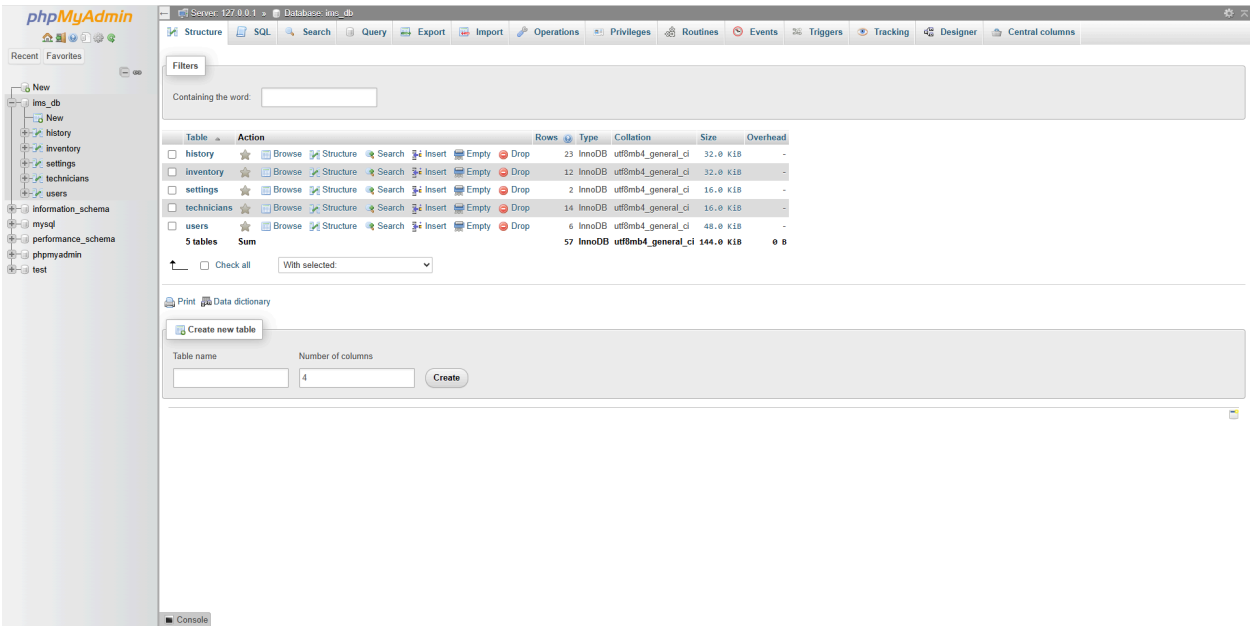
- **Frontend:** Dynamic UI rendering using PHP to automatically reflect current database column names and structures.
- **Backend:** PHP environment featuring a dynamic database handler that adapts to schema changes without manual code updates.
- **Database:** MySQL (**ims_db**) for centralized storage, integrated via a unified configuration file for consistent connectivity.
- **Synchronization:** AJAX-enabled 'Sync Changes' feature for seamless bulk updates through a modal interface without page reloads.

Database Schema **(on going)**

The system utilizes four primary collections. Below is the structure for the `Product` and `Transaction` models:

***Product Model*:** | Field | Type | Description | | :--- | :--- | :--- |

```
└── OJTProject/
    ├── index.php
    ├── script.js
    ├── style.css
    ├── include/
    │   ├── auth_checker.php
    │   └── config.php
    ├── component/
    │   ├── search.js
    │   ├── sort.js
    │   ├── settings/
    │   │   ├── addUser.css
    │   │   ├── addUser.js
    │   │   ├── logout.php
    │   │   ├── settings.css
    │   │   └── settings.php
    │   ├── selectMonth/
    │   │   ├── selectMonth.css
    │   │   └── selectMonth.php
    │   ├── qrGen/
    │   │   ├── qr_gen.html
    │   │   └── qr_gen.php
    │   ├── navbar/
    │   │   ├── nav-bar.css
    │   │   └── nav-bar.php
    │   ├── inventoryTableHandler/
    │   │   ├── inventoryTableHandler.css
    │   │   └── inventoryTableHandler.php
    │   ├── inventoryModal/
    │   │   ├── inventoryModal.css
    │   │   └── inventoryModal.php
    │   ├── inventoryAlertBox/
    │   │   ├── inventoryAlerts.css
    │   │   └── inventoryAlerts.php
    │   ├── graphBox/
    │   │   ├── chart.js
    │   │   ├── graph.css
    │   │   └── graph.php
    │   └── currency/
    │       ├── currency.css
    │       └── currency.php
    └── pages/
        ├── signup/
        │   └── signup.php
        ├── inventory/
        │   ├── inventory.css
        │   ├── inventory.js
        │   ├── inventory.php
        │   └── inventoryFunction.php
        ├── history/
        │   └── history.php
        └── dashBoard/
            ├── dashBoard.css
            └── dashBoard.php
```



Appendix

1. Project Overview & Scope

This section sets the stage. It’s for stakeholders to understand the "why" and the "what."

- **Executive Summary:** A high-level brief of what the system does.
- **Objectives:** (e.g., "Reduce stockouts by 20%" or "Automate reorder points").
- **Tech Stack:** The languages, databases (like PostgreSQL or MongoDB), and frameworks used.
- **Key Features:** A bulleted list of core functionalities (Barcode scanning, reporting, etc.).

2. Technical Documentation

This is for the developers. It explains the "engine" under the hood.

- **System Architecture:** A diagram or description of how the frontend, backend, and database interact.
- **Database Schema:** Definitions of tables (Products, Suppliers, Transactions, Users) and their relationships.
- **API Documentation:** If the system connects to other tools, list the endpoints and request/response formats.
- **Installation Guide:** Step-by-step instructions on how to set up the environment and run the code.

3. Functional Requirements & Logic

This is the most critical part—it defines the "rules" of your inventory.

- **User Roles & Permissions:** Who can view stock vs. who can delete records (Admin, Manager, Staff).
- **Inventory Logic: * Valuation Method:** Are you using FIFO (First-In, First-Out), LIFO, or Weighted Average?
    - **Reorder Logic:** The mathematical formula used to trigger alerts.

  For example, a simple reorder point R can be calculated as:

  $$R = (d \times L) + s$$

  Where d is average daily usage, L is lead time, and s is safety stock.
- **Workflow Diagrams:** Flowcharts showing how an item moves from "Received" to "Sold."

4. User Manual & Maintenance

This is for the daily users (the warehouse team).

- **Getting Started:** How to log in and navigate the dashboard.

- **Common Tasks:** "How to add a new SKU," "How to generate a monthly report."
- **Troubleshooting:** A FAQ for common errors (e.g., "Duplicate SKU detected").
- **Data Backup & Security:** How the data is saved and kept safe.

- **Project Title:**
- Hayakawa Jig IMS Development
- **Reporting Period:**
- Initial Phase / Current Architecture
- **Tech Stack:**
- PHP, JavaScript, CSS, HTML, MySQL (XAMPP)
- **1. Project Overview and Problem Statement:**
- The Jig Department currently faces challenges in maintaining accurate records of specialized tools and materials.
  The absence of a dedicated digital system has led to manual tracking, increasing the risk of stock inaccuracies and limiting data accessibility.
  The objective of this project is to develop a lightweight, high-performance web-based Inventory Management System (IMS)
  to centralize inventory data, support dynamic database schema updates, and provide a user-friendly interface for department staff.
- **2. Current System Architecture:**
- The system is implemented using a standard WAMP environment through XAMPP. It incorporates a dynamic database handler
  that enables the application to automatically adapt when new columns are added to the MySQL database without requiring manual changes to the frontend code.

  Key architectural components include:
  • Database Integration – Connection to a MySQL database named 'ims_db' via a centralized configuration file.
  • Dynamic UI Rendering – The frontend retrieves column names directly from the database schema using PHP to ensure the interface reflects the current structure.
  • AJAX Synchronization – A 'Sync Changes' feature allows bulk updates through a modal interface without requiring a full page reload.
-
- **3. Progress Summary and Implemented Functionality:**
- • Database Connectivity – A functional database connection script has been implemented with error handling.

• Inventory Display – The system dynamically renders inventory data, including category, item name, description, quantity, and price.
• Automated Calculations – A 'Total' column is automatically computed as quantity multiplied by price.
• Addition Logic – New entries are inserted using duplicate-key logic to increment quantity when an item already exists.
• Bulk Management – Users can edit multiple rows simultaneously and delete selected entries.
• Dynamic Column Creation – The backend automatically detects and creates new database columns when required.

- **4. System Operational Flow:**
- 1. The user opens the inventory page through a web browser.
  2. PHP retrieves inventory records and column metadata using SELECT and SHOW COLUMNS queries.
  3. The system dynamically builds the inventory table interface.
  4. The user selects 'Edit' to open a modal containing an editable copy of the table.
  5. The user modifies values directly within editable cells.
  6. Upon selecting 'Sync Changes', the updated data is sent to the server using AJAX in JSON format.
  7. The backend validates columns, sanitizes inputs, and updates the MySQL database accordingly.
  8. The interface refreshes to reflect the latest data.
- **5. System Logic Overview (Pseudo-Code):**
- Frontend Logic (JavaScript):
  • Open modal when 'Edit' is clicked.
  • Clone inventory table into modal view.
  • Enable editable mode for allowed cells.
  • Collect edited data and convert to JSON.
  • Send POST request to backend.
  • Refresh interface after successful update.

  Backend Logic (PHP):
  • Verify column existence and create if needed.
  • Insert new items using duplicate-key logic.
  • Iterate through JSON update data.
  • Sanitize and validate inputs.
  • Execute UPDATE queries for each modified row.
- **6. Planned Improvements and Next Steps:**
- • History Logging – Implement a logging system to track all inventory modifications.
  • Reporting Tools – Add export features such as PDF generation and Excel export.
  • User Authentication – Implement a login using password hashing and session management.
  • Access Control – Restrict editing privileges based on user roles.
-